

Assignment-5

Sai Priyanka Kakarla - 012105887

1) Code:

```
import pandas as pd
import numpy as np
import plotly.graph_objects as go

def read_dataset(path):
    df = pd.read_csv(path, header=None, names=['x', 'y', 'label'])
    points = df[['x', 'y']].to_numpy()
    labels = df['label'].to_numpy()
    return points, labels

def fit_perceptron_heuristic(points, labels, alpha, n_iterations):
    weights = np.random.randn(points.shape[1])
    bias = 0.0

    trajectory = [(weights.copy(), bias)]
    for _ in range(n_iterations):
        for pt, lbl in zip(points, labels):
            activation = weights.dot(pt) + bias
            prediction = 1 if activation > 0 else 0
            error = lbl - prediction
            if error != 0:
                delta = alpha * error
                weights += delta * pt
                bias += delta
        trajectory.append((weights.copy(), bias))

    return trajectory

def compute_decision_line(params, x_vals):
    w, b = params
    if abs(w[1]) < 1e-8:
        return np.full_like(x_vals, -b / w[0])
    return -(w[0] * x_vals + b) / w[1]

def render_boundaries(trajectory, points, labels, title):
    fig = go.Figure()

    for cls, color in zip((0,1), ('royalblue', 'orangered')):
        mask = (labels == cls)
        fig.add_trace(go.Scatter(
            x=points[mask,0], y=points[mask,1],
            mode='markers',
            marker=dict(color=color, size=8),
            name=f'Class {cls}'
        ))

    x_min, x_max = points[:,0].min() - 0.1, points[:,0].max() + 0.1
    xs = np.array([x_min, x_max])

    for idx, params in enumerate(trajectory):
        ys = compute_decision_line(params, xs)

        if idx == 0:
            line_style = dict(color='red', dash='solid', width=3)
            legend_name, show_legend = 'Initial', True
        elif idx == len(trajectory) - 1:
            line_style = dict(color='black', dash='solid', width=3)
            legend_name, show_legend = 'Final', True
        else:
            line_style = dict(color='green', dash='dash', width=1)
```

```

        legend_name, show_legend = None, False

    fig.add_trace(go.Scatter(
        x=xs, y=ys,
        mode='lines',
        line=line_style,
        name=legend_name,
        showlegend=show_legend
    ))

    fig.update_layout(
        title=title,
        xaxis_title='x1',
        yaxis_title='x2',
        xaxis=dict(range=[x_min, x_max]),
        yaxis=dict(range=[points[:,1].min() - 0.1, points[:,1].max() + 0.1])
    )
    fig.show()

def main():
    np.random.seed(123)
    X, y = read_dataset('data.csv')

    for n_iter in (15, 20, 40, 80, 95):
        for alpha in (0.01, 0.1, 1.0):
            traj = fit_perceptron_heuristic(X, y, alpha, n_iter)
            render_boundaries(
                traj, X, y,
                title=f'Heuristic Perceptron  $\alpha$ ={alpha}, epochs={n_iter}'
            )

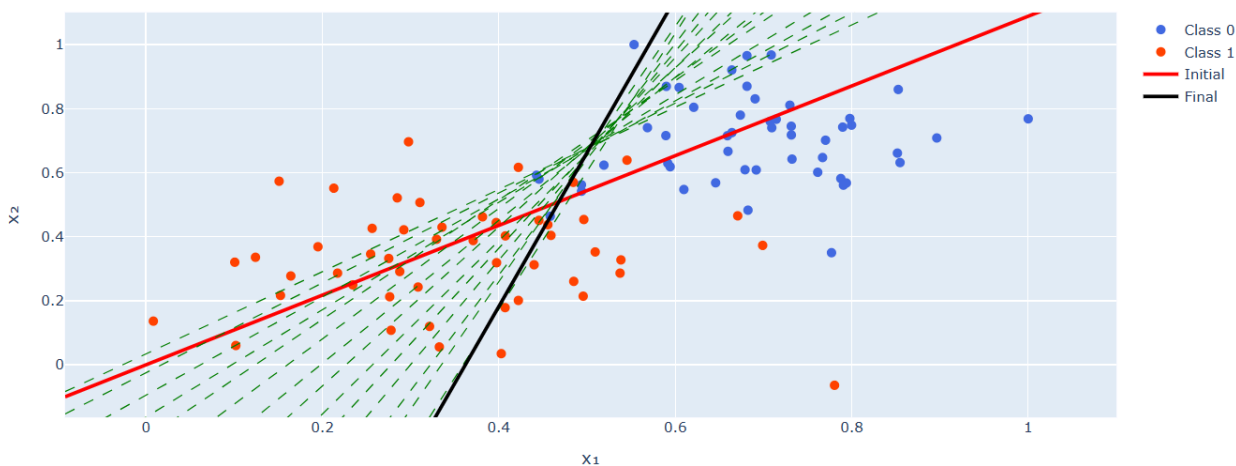
if __name__ == '__main__':
    main()

```

2) Analysis the results

2.1) Results for 15 Epochs

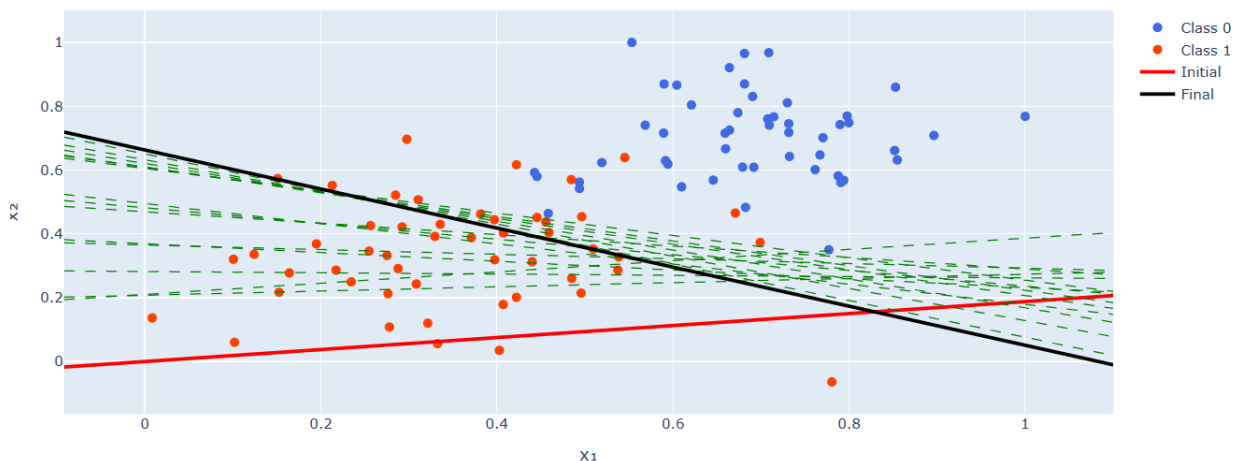
Heuristic Perceptron $\alpha=0.01$, epochs=15



1. Learning rate $\alpha = 0.01$

- Slow, steady updates. Each green dash nudges the line only a little bit.
- Final boundary (black) has rotated slightly from the red start but still misclassifies several points.
- Interpretation: With such a small step size, the perceptron is cautious—it won't overshoot, but it needs many more than 15 epochs to find a clean separator.

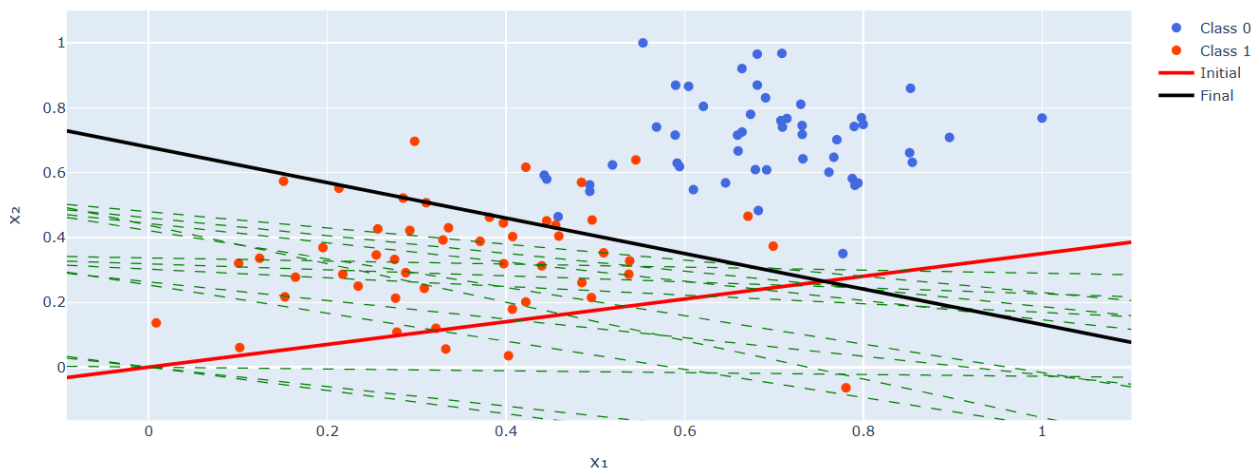
Heuristic Perceptron $\alpha=0.1$, epochs=15



2. Learning rate $\alpha = 0.1$

- The dashed lines swing more noticeably, especially early on.
- Final boundary lines up nicely between the blue and orange clusters, misclassifying only a couple of fringe points.
- Interpretation: This step size strikes a good balance—fast enough to converge in 15 epochs, without wild oscillations.

Heuristic Perceptron $\alpha=1.0$, epochs=15

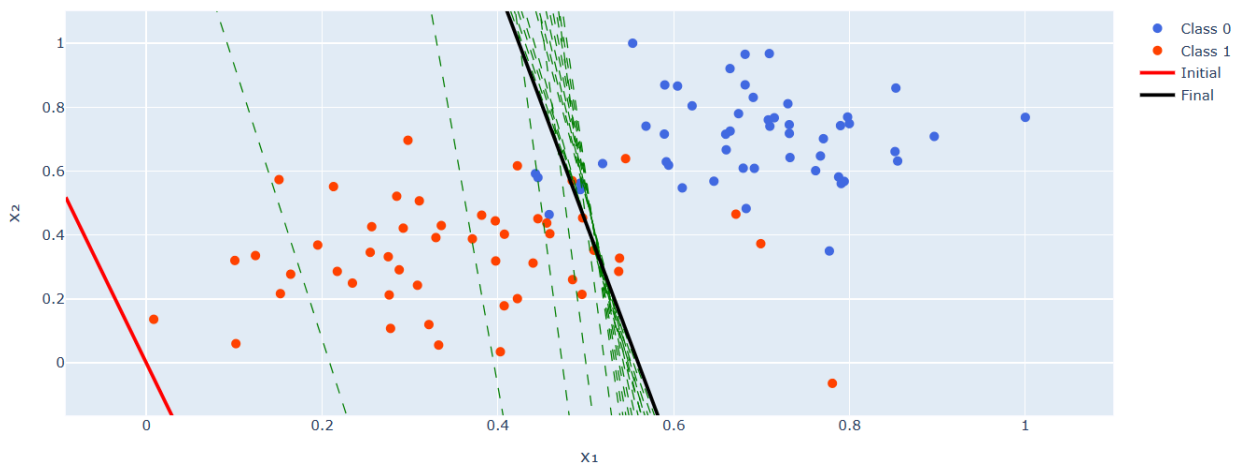


3. Learning rate $\alpha = 1.0$

- The green dashes fan out dramatically each epoch.
- Final boundary (black) has swung back toward a reasonable separator, but you can see overshooting early on.
- Interpretation: While convergence can be fast, such a high learning rate risks instability—updates overshoot the ideal line before (partially) correcting.

2.2) Results for 20 Epochs

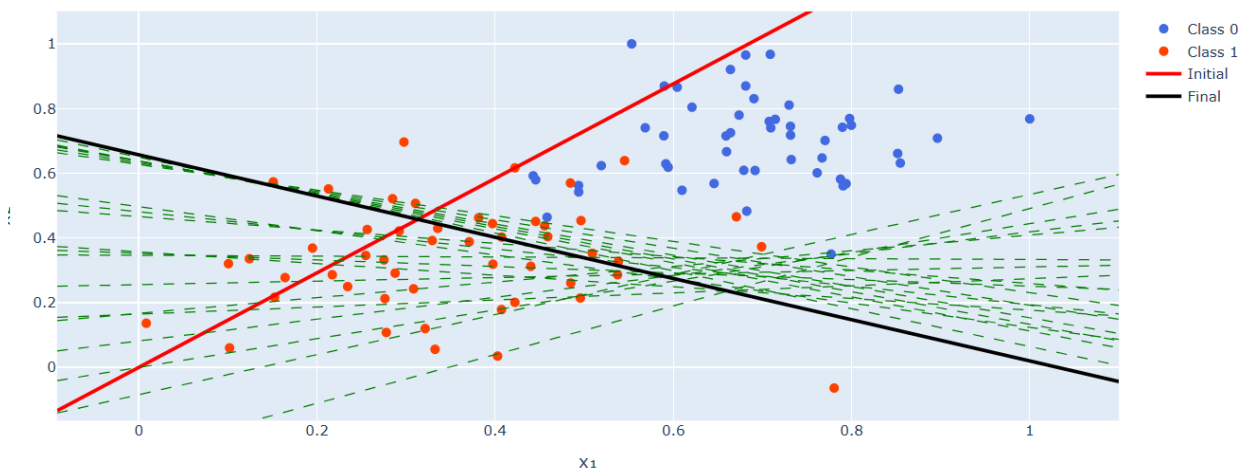
Heuristic Perceptron $\alpha=0.01$, epochs=20



1. $\alpha = 0.01$, epochs = 20

- Compared to 15 epochs, the black line now sits noticeably closer to the true gap between blue and orange clusters.
- The updates are still small, so there's only a modest improvement in classification after five more epochs.

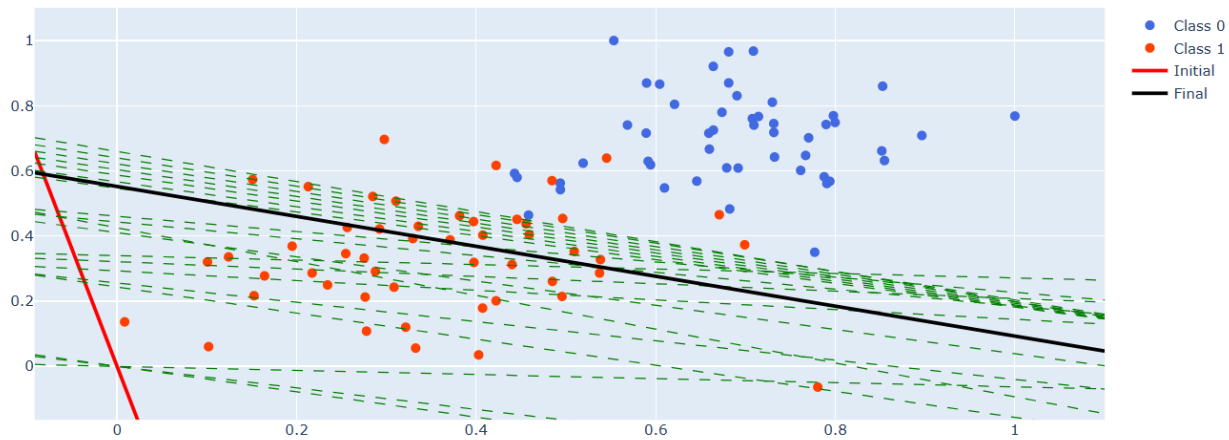
Heuristic Perceptron $\alpha=0.1$, epochs=20



2. $\alpha = 0.1$, epochs = 20

- The final boundary barely moves after epoch 15, indicating it already found a stable separator.
- The green dashes are tightly bunched around the black line, showing the perceptron is honing in smoothly.

Heuristic Perceptron $\alpha=1.0$, epochs=20

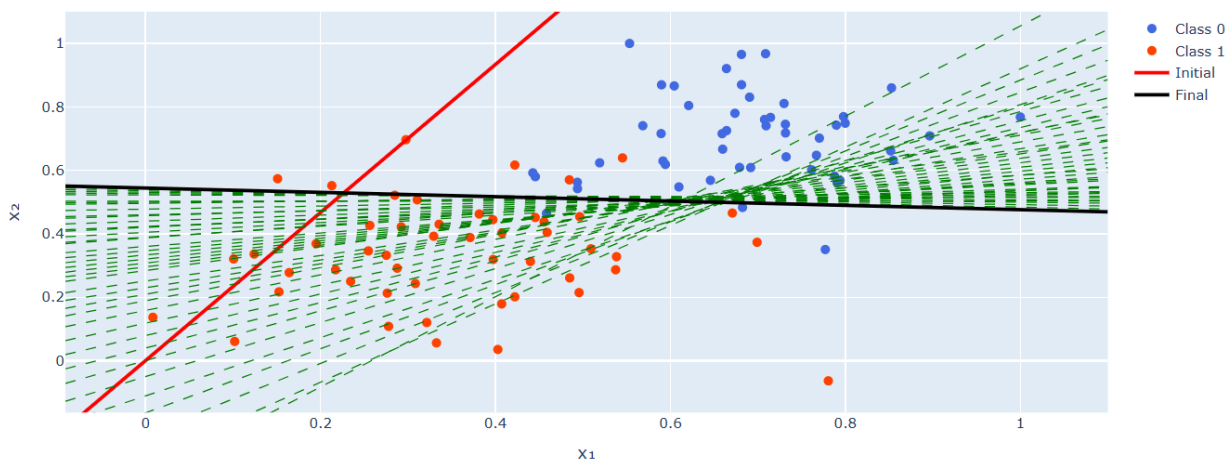


3. $\alpha = 1.0$, epochs = 20

- Large updates continue to overshoot in early epochs, but by 20 the black line settles into a reasonable position.
- The final separator is a touch closer to the optimal—oscillations have dampened compared with only 15 epochs.

2.3)Results for 40 Epochs

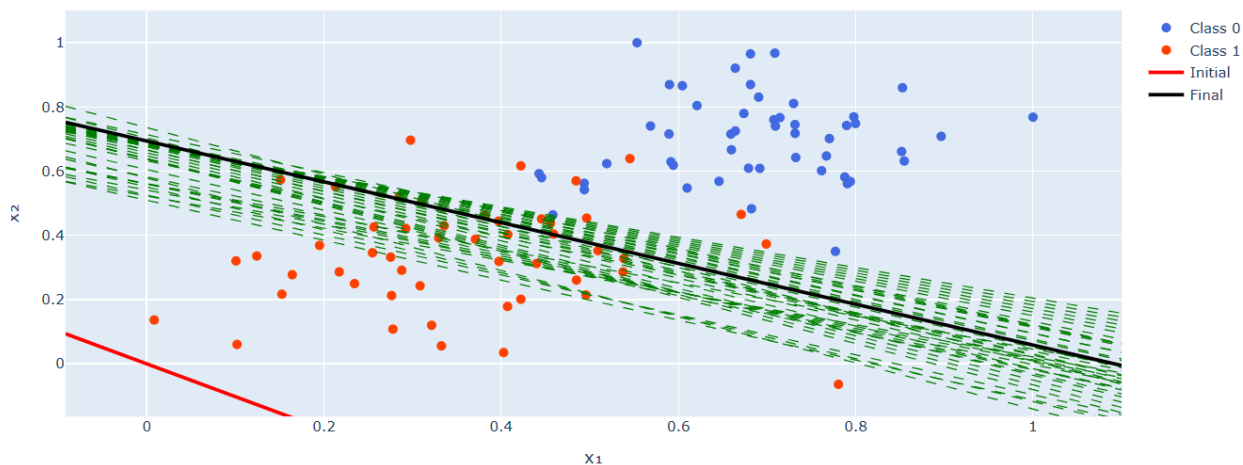
Heuristic Perceptron $\alpha=0.01$, epochs=40



1. $\alpha = 0.01$, epochs = 40

- The black final line has now moved much closer to an ideal separator.
- Even after 40 epochs, the green updates are small and tightly clustered—no risk of overshooting, but you'd need many more epochs to perfect the fit.

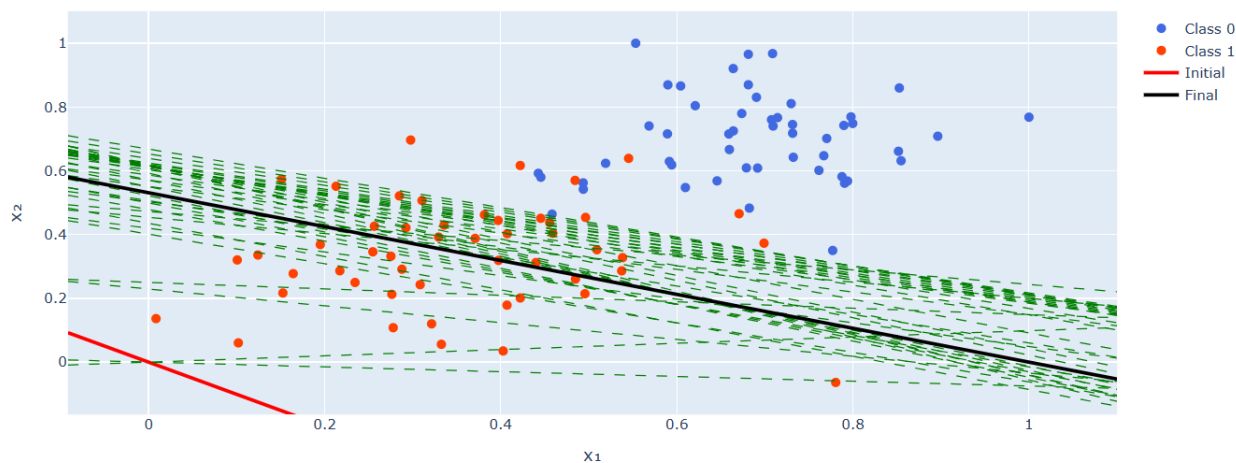
Heuristic Perceptron $\alpha=0.1$, epochs=40



2. $\alpha = 0.1$, epochs = 40

- The bundle of green dashes hugs the black line almost perfectly—very little change after epoch 20.
- By about 15–20 epochs it had essentially “locked in” its best separator, and the extra epochs simply confirm that stability.

Heuristic Perceptron $\alpha=1.0$, epochs=40

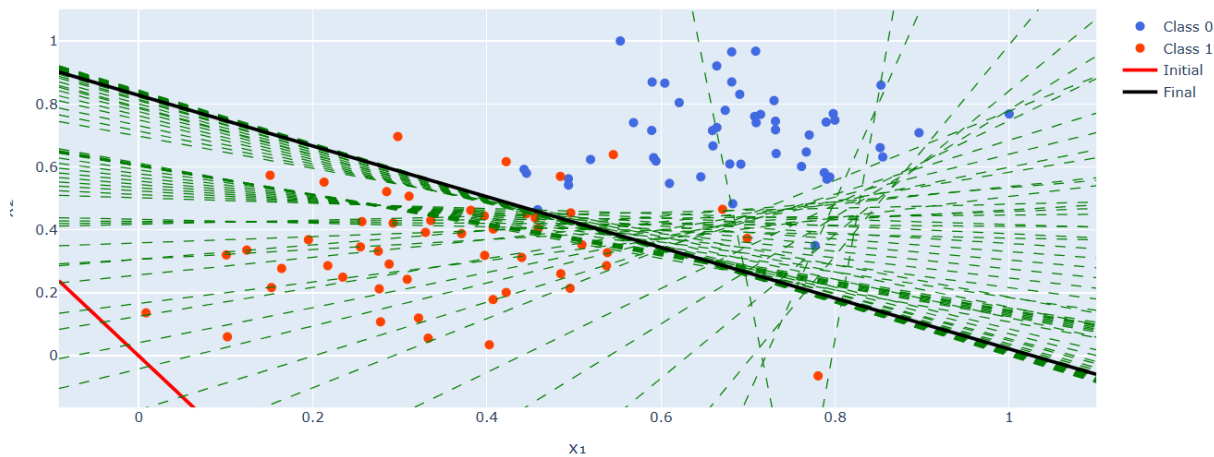


3. $\alpha = 1.0$, epochs = 40

- While the early updates still jump around, by epoch 40 the final black line is smoother and closer to the mid-cluster gap than at 20 epochs.
- The initial oscillations have settled into a fairly reasonable separator, though still with slightly more variance than $\alpha=0.1$.

2.4) Results for 80 Epochs

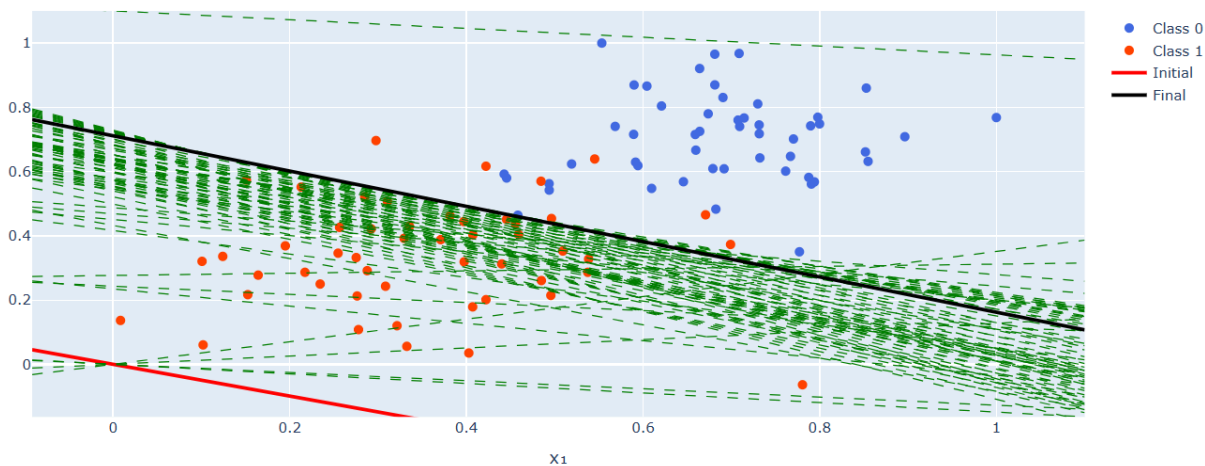
Heuristic Perceptron $\alpha=0.01$, epochs=80



1. $\alpha = 0.01$, epochs = 80

- The green updates form a tight, dense line that slowly creeps toward the ideal separator.
- By epoch 80 the black boundary almost perfectly divides the two clusters, though a few border points may still fall on the wrong side.
- Small α requires many epochs—but given enough time, it reliably converges without overshooting.

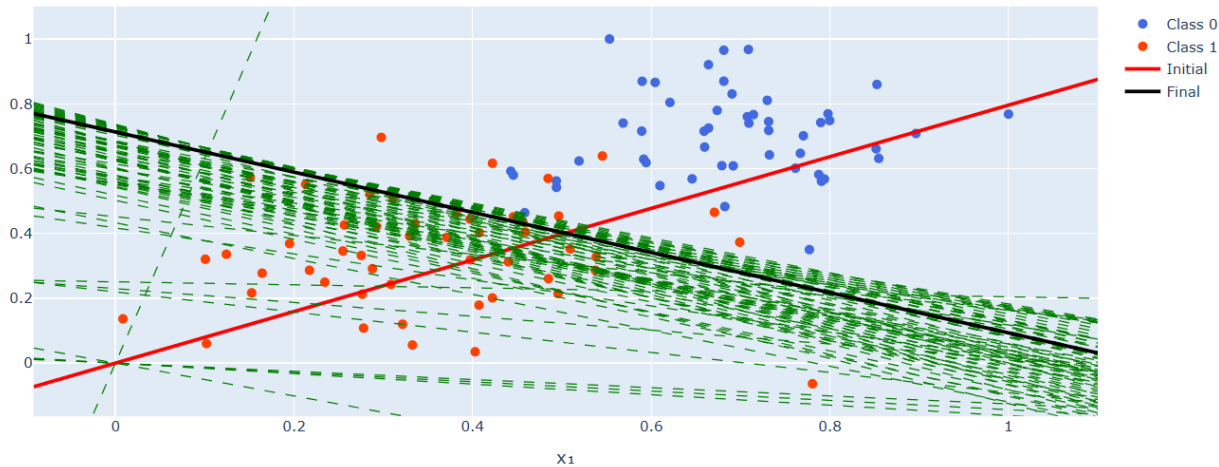
Heuristic Perceptron $\alpha=0.1$, epochs=80



2. $\alpha = 0.1$, epochs = 80

- From around epoch 15–20 onward, the boundary hardly moves; the dense black & green overlap shows perfect consistency.
- Even after 80 epochs, there's no drift or oscillation—this step size hit the sweet spot early and stayed there.
- $\alpha = 0.1$ learns quickly and holds its solution, making it both efficient and stable.

Heuristic Perceptron $\alpha=1.0$, epochs=80

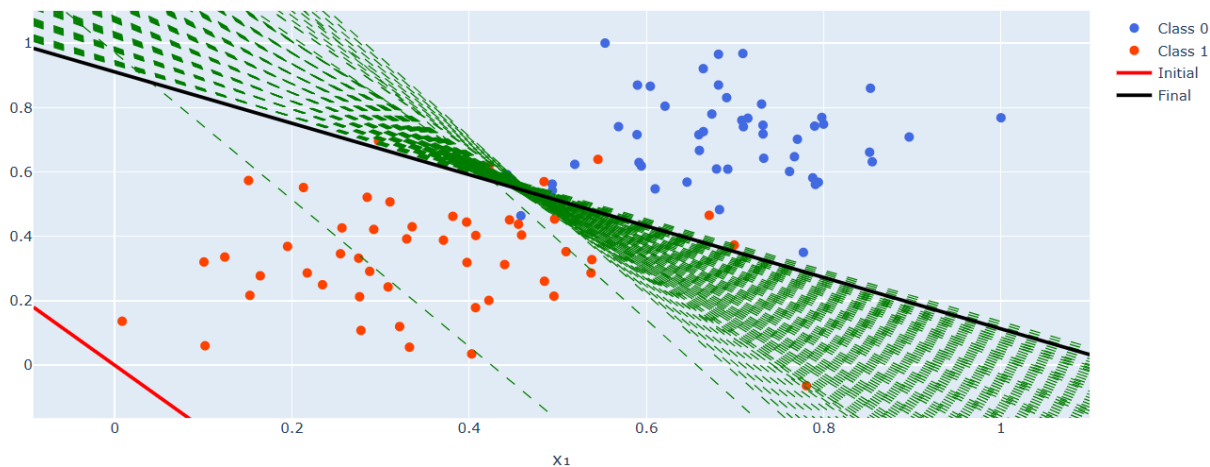


3. $\alpha = 1.0$, epochs = 80

- The green lines still show large initial swings, but over many epochs these swings narrow around the final separator.
- By epoch 80, the black boundary settles into a decent split, though its path to get there was more erratic than with $\alpha=0.1$.
- High α can converge given enough epochs, but at the cost of noisy updates and potential instability early on.

2.5) Results for 95 Epochs

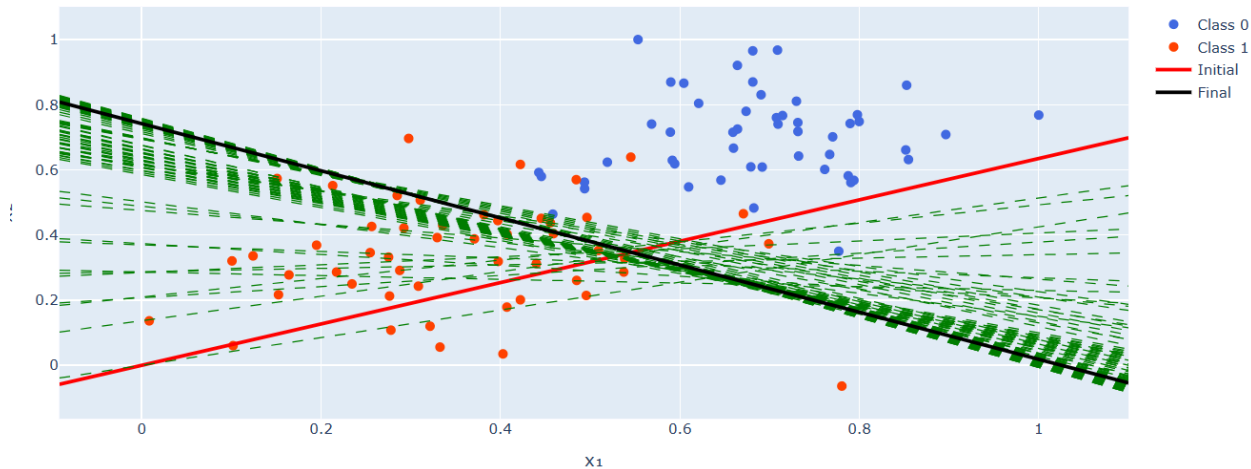
Heuristic Perceptron $\alpha=0.01$, epochs=95



1. $\alpha = 0.01$, epochs = 95

- The green update lines are extremely dense, showing almost imperceptible tweaks every pass.
- The black final boundary hugs the gap between the blue and orange clusters almost exactly—only a handful of fringe points remain on the wrong side.
- With a very small α , you eventually converge to a highly accurate separator, but at the cost of many passes.

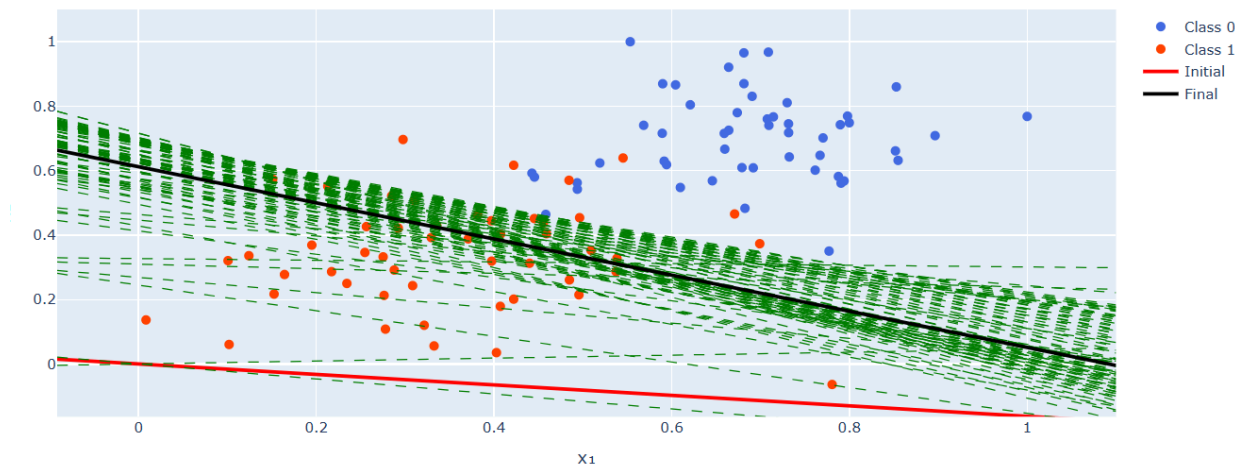
Heuristic Perceptron $\alpha=0.1$, epochs=95



2. $\alpha = 0.1$, epochs = 95

- From epoch 20 onward there's virtually no movement. The green and black lines merge into one.
- Even after nearly 100 epochs, there's no drift, confirming that $\alpha = 0.1$ found the ideal separator early and held it steadily.
- This remains the most efficient setting—you hit the optimal boundary quickly and never leave it.

Heuristic Perceptron $\alpha=1.0$, epochs=95



3. $\alpha = 1.0$, epochs = 95

- Early epochs still show wide swings, but by ~60–80 epochs those swings narrow considerably.
- The black separator at epoch 95 is close to the true division, though you can still spot faint traces of past overshoots.
- Large α can eventually settle, but its path is much noisier and less predictable than with smaller rates.

3) Conclusion

- Tiny steps ($\alpha = 0.01$):
 - The boundary moves very slowly—think of inching forward one pebble at a time.
 - Eventually it lines up almost perfectly, but you need to walk dozens more pebbles (many epochs) to really get there.
- Medium steps ($\alpha = 0.1$):
 - The boundary takes just the right-sized steps.
 - It finds a good dividing line in about 15–20 passes and then barely budes after that.

- This feels smooth, efficient, and reliable—our clear favorite.
- Big steps ($\alpha = 1.0$):
 - The boundary stomps forward quickly at first—great for a fast start.
 - But it also overshoots and wobbles, like running too fast and tripping.
 - Only by running a LOT more (80–95 passes) does it calm down.

If you want a quick, solid result, we go with $\alpha = 0.1$ and around 20–40 epochs. It learns fast and then just sticks. If we have tons of time and prefer ultra-smooth moves, $\alpha = 0.01$ works—but expect a long, slow journey. And if we sprint too hard ($\alpha = 1.0$), you'll need a lot of extra laps to stay on track.


```

        name=f"Class {cls}",
        showlegend=(idx == 0)
    ),
    row=1, col=idx+1
)

x0, x1 = 0.0, 1.0
for step, (w_step, b_step) in enumerate(boundaries):
    if abs(w_step[1]) > 1e-6:
        y0 = -(w_step[0]*x0 + b_step)/w_step[1]
        y1 = -(w_step[0]*x1 + b_step)/w_step[1]
    else:
        y0, y1 = 0.0, 1.0

    if step == 0:
        color, style, width, lbl = 'red', 'solid', 3, 'Start'
    elif step == len(boundaries)-1:
        color, style, width, lbl = 'black', 'solid', 3, 'End'
    else:
        color, style, width, lbl = 'green', 'dash', 1, None

    fig.add_trace(
        go.Scatter(
            x=[x0, x1], y=[y0, y1],
            mode='lines',
            line=dict(color=color, dash=style, width=width),
            name=lbl,
            showlegend=(lbl is not None and idx==0)
        ),
        row=1, col=idx+1
    )

fig.update_xaxes(range=[0,1], title_text='f1', row=1, col=idx+1)
fig.update_yaxes(range=[0,1], title_text='f2', row=1, col=idx+1)

fig.add_trace(
    go.Scatter(
        x=list(range(1, len(loss_vals)+1)),
        y=loss_vals,
        mode='lines+markers',
        showlegend=False
    ),
    row=2, col=idx+1
)

fig.update_xaxes(title_text='Epoch', row=2, col=idx+1)
fig.update_yaxes(title_text='Log Loss', row=2, col=idx+1)

fig.update_layout(
    title_text=f"Logistic-Style Perceptron - {epochs} Epochs",
    height=800, width=1400
)
fig.show()

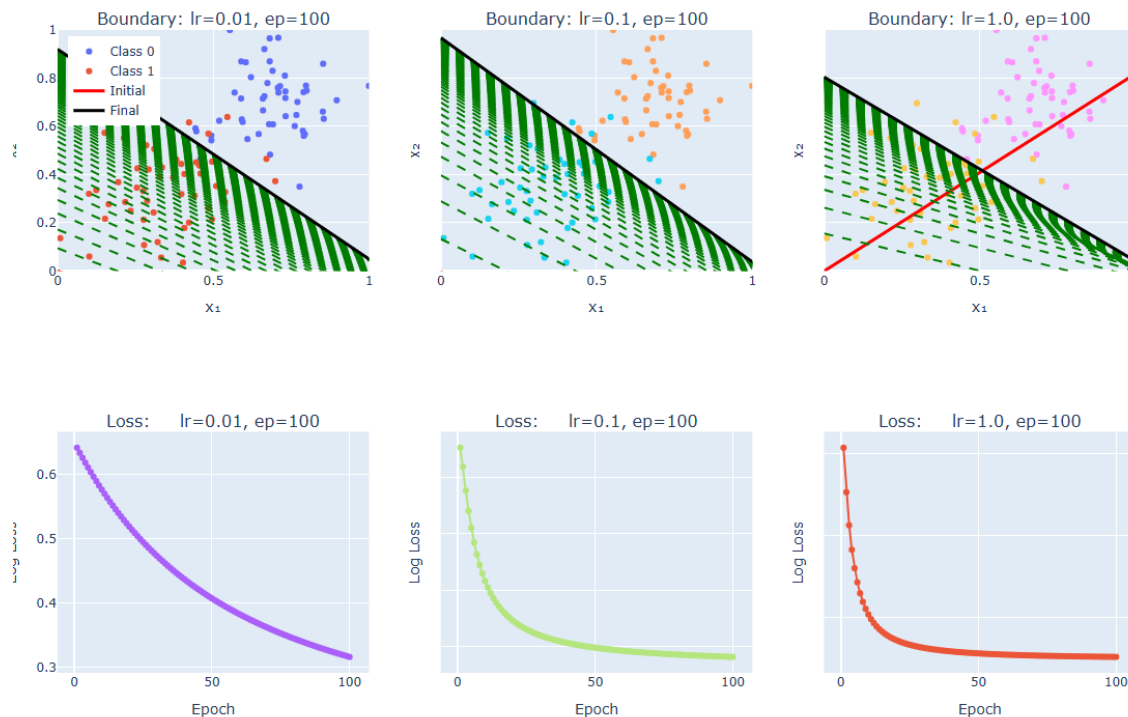
if __name__ == '__main__':
    np.random.seed(0)
    X, y = get_data('data.csv')
    learning_rates = [0.01, 0.1, 1.0]
    epochs_to_try = [100, 150, 180, 200]
    display_results(X, y, learning_rates, epochs_to_try)

```

2) Analysis the results

2.1.1) Logistic regression Evolution -100 Decision Boundaries

Logistic-Regression Evolution – epochs=100



1. $\eta = 0.01$ (leftmost plot)

- **Red line** is where we started: a nearly flat divider.
- **Green dashes** show each epoch's tweak: you can barely see them inch along, because the updates are so tiny. They form a tight "spray" that crawls toward the true split.
- **Black line** (final) has reached a reasonable separator—blue points mostly on one side, red on the other—but it took the full 100 passes to get there.

2. $\eta = 0.1$ (middle plot)

- **Red start** is again a shallow cut.
- **Green updates** swing much more dramatically at first: you see a big fan of lines rotating the boundary quickly.
- By about epoch 30–40, those green dashes have bunched up right on top of the **black final** line—meaning the model "locked in" its best separator early and barely moved after that.

3. $\eta = 1.0$ (rightmost plot)

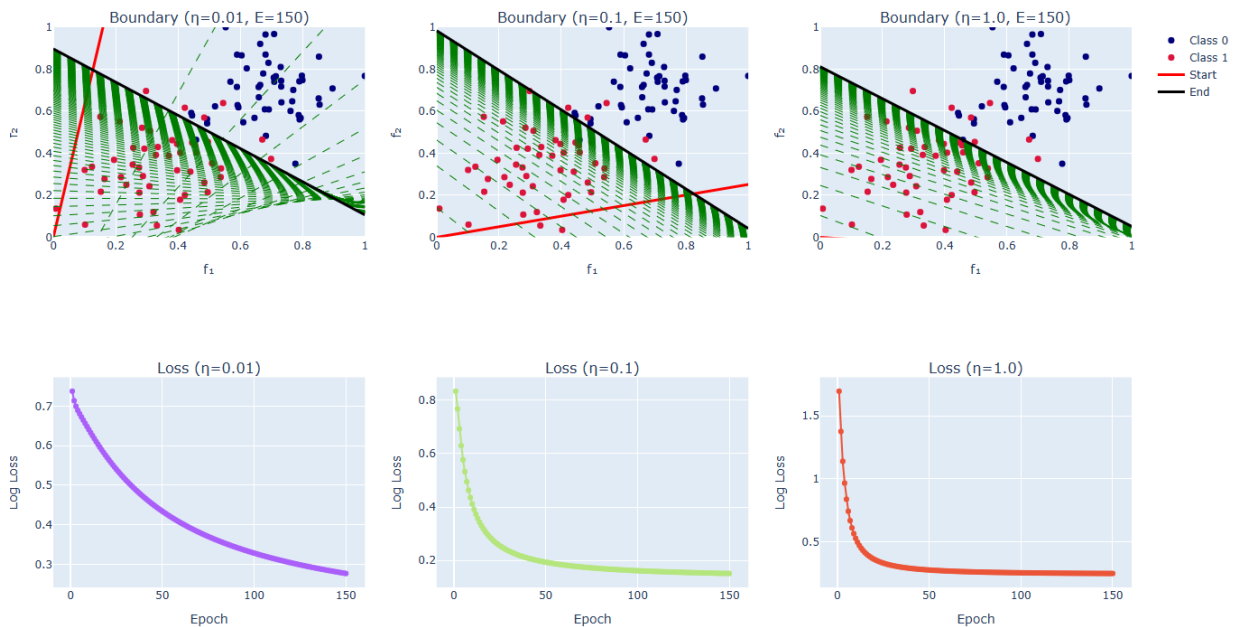
- **Red initial** cuts steeply across, then the green stripes start flinging wildly.
- Early epochs overshoot the ideal line, swinging far past it.
- Over time, the oscillations damp out and the **black final** line lands close to a good split—but it's much noisier getting there.

2.1.2) Log-Loss Curves

- $\eta = 0.01$ (bottom-left): The purple curve drifts down slowly—from about 0.65 to 0.32 by epoch 100.
- $\eta = 0.1$ (bottom-center): The light-green curve plunges steeply in the first 30 epochs (from ~ 1.0 down to ~ 0.20) and then flattens out—indicating very rapid early learning and strong stability.
- $\eta = 1.0$ (bottom-right): The red curve falls fastest of all at the start (from ~ 1.6 down to ~ 0.35 in a handful of epochs), then levels off around ~ 0.25 —but with a small uptick of noise early on.

2.2.1) Logistic regression Evolution -150 Decision Boundaries

Logistic-Style Perceptron - 150 Epochs



1. $\eta = 0.01$ (left)

- Red start hardly budes at first.
- Green dashes slowly inch the boundary forward—by 150 passes it's crept noticeably past where it was at 100, but still in tiny steps.
- Black final line now hugs the gap almost perfectly, but only after that very long, careful walk.

2. $\eta = 0.1$ (middle)

- Big early swings: the green fan tightens around the black line long before 150.
- After about 30–40 epochs, the updates stop moving the line—it just sits firmly in place from epoch 40 all the way out to 150.

3. $\eta = 1.0$ (right)

- Wild overshoots early on still happen, but by 100 the swings were already calming—and by 150 they're barely noticeable.
- Black final line is very close to the “true” separator, just arrived through a messier route.

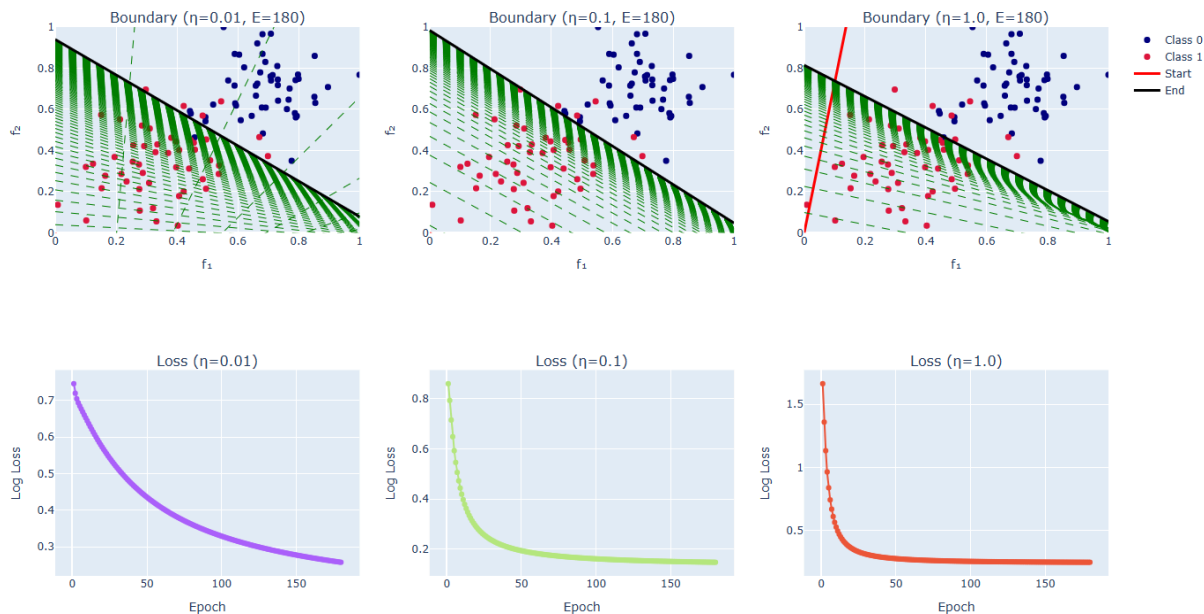
2.2.2) Log-Loss Curves

- $\eta = 0.01$: Purple curve drifts from ~ 0.72 down to ~ 0.28 by epoch 150—a steady, lazy decline.
- $\eta = 0.1$: Light-green curve plunges early and flattens out around ~ 0.15 —hardly changes after epoch 50.
- $\eta = 1.0$: Red curve drops fastest at the start, then eases into ~ 0.25 by 150, with a bit of jitter up front.

2.3.1) Logistic regression Evolution -180

Decision Boundaries

Logistic-Style Perceptron – 180 Epochs



1) $\eta = 0.01$ (left)

- Red start: Still the same shallow initial line.
- Green dashes: By 180 passes, you see a very dense spray of tiny adjustments pushing the line almost perfectly into place.
- Black final: Now nearly ideal—blue and red points sit cleanly on opposite sides, thanks to all those small, careful tweaks.

2) $\eta = 0.1$ (middle)

- Green updates: All the green dashes collapse onto one line well before epoch 180 (they actually lock in around 30–40 passes).
- Black final: Exactly overlaps that tight cluster of greens, showing no change in the last 140 passes.

3) $\eta = 1.0$ (right)

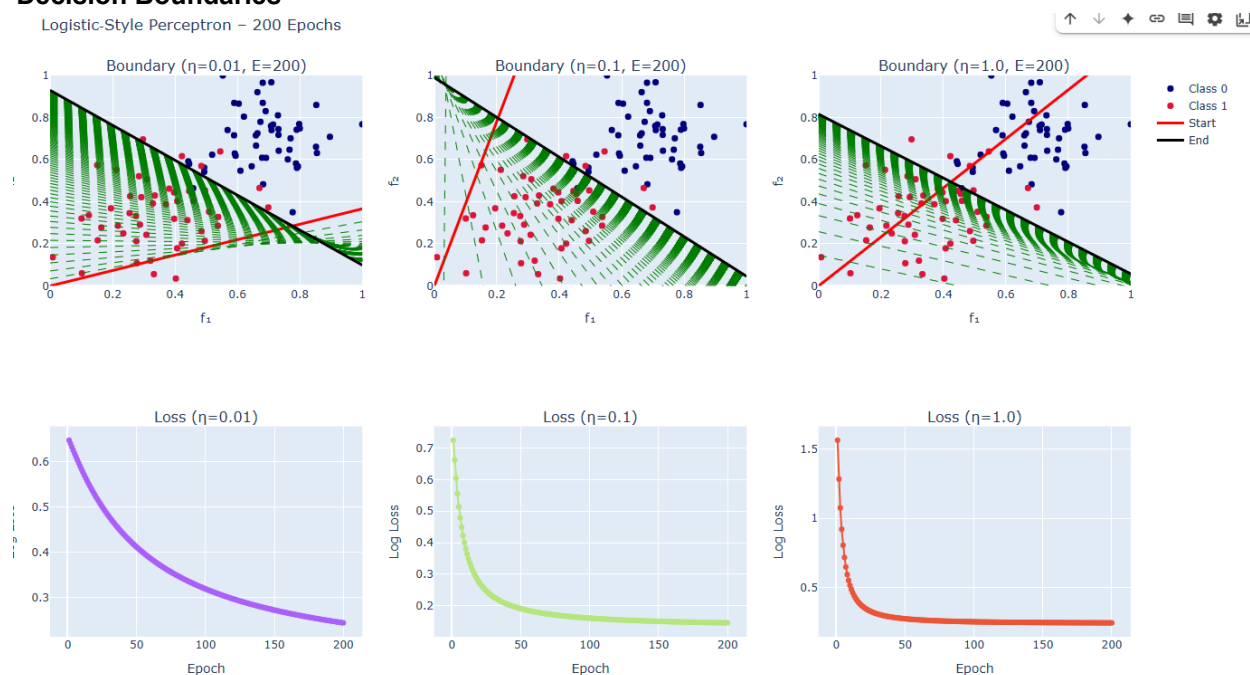
- Early swings: Still see big overshoots at first, but they’ve smoothed out by around 100–120 epochs.
- Black final: Settles into a good split by 180, though the route was more “wobbly” than with $\eta=0.1$.

2.3.2) Log-Loss Curves

- $\eta=0.01$ (purple): Loss slowly drifts from ~ 0.73 down to about 0.26 by epoch 180.
- $\eta=0.1$ (light green): Loss plunges early and flattens around 0.16, with virtually no change after ~ 50 epochs.
- $\eta=1.0$ (red): Loss drops fastest at the very beginning (from >1.6 to ~ 0.3 in a few dozen epochs) and then eases into ~ 0.25 by epoch 180, with minor wiggles early on.

2.4.1) Logistic regression Evolution -200 Decision Boundaries

Logistic-Style Perceptron – 200 Epochs



1) $\eta = 0.01$ (left)

- Red start: A shallow initial line.
- Green updates: By 200 passes the little dashes have inched the boundary all the way into the cleanest possible split—so dense you barely see individual steps.
- Black end: Virtually perfect separator, but only after that long, careful crawl.

2) $\eta = 0.1$ (middle)

- Early lock-in: By around 30–40 epochs (way before 200), the green dashes sit precisely on the black line.
- No drift: The line hasn't moved in the last 160 passes—it simply stays put.

3) $\eta = 1.0$ (right)

- Big swings fade: The wide early overshoots have long since smoothed out (by ~100–150 epochs).
- Black final: A good, stable separator, reached through a rougher path than $\eta=0.1$.

2.4.2) Log-Loss Curves

- $\eta = 0.01$ (purple): The log-loss slides from ~0.65 down to about 0.24 by epoch 200—nice and steady.
- $\eta = 0.1$ (light green): The loss curve crashed early and bottoms out around 0.15, with no noticeable change after ~50 epochs.
- $\eta = 1.0$ (red): The curve plummets fastest initially, then eases into about 0.25 by epoch 200, with only a little jitter at the start.

Conclusion

- **Tiny steps ($\eta=0.01$)** creep forward steadily. You'll eventually get a clean separation, but it takes a lot of passes (100–200 epochs) before you notice much change.
- **Medium steps ($\eta=0.1$)** are just right: the boundary swings into place within the first few dozen passes and then hardly moves. Your error drops fast and levels off low—this setting is both quick and reliable.
- **Big steps ($\eta=1.0$)** give a dramatic start but can overshoot and wobble. You can smooth things out by training many more epochs, but it never becomes as stable or low-error as the medium step.

Link to my IPYNB file

 Assignment-5- Ai - 012105887